

Estruturas de dados em Python: de “lista pra tudo” a um sistema de estoque

Resumo da pesquisa feita durante a prática "O Especialista em Python e o Iniciante que “Usa Lista pra Tudo”": lista, tupla, set e dicionário, por que a busca em lista é lenta, o erro "unhashable", e um controle de estoque construído passo a passo (cadastro, baixa, relatório, menu, JSON, try/except, orientação a objetos, módulos, histórico de vendas e interface com Tkinter), além de laços e list comprehension.

Fernando Melo · 5 de outubro de 2026 · Pesquisa no Modo IA do Google, 09:49

01 — LISTA, TUPLA, SET E DICIONÁRIO

Pergunta

"Opa! Eu tô aprendendo Python e, confesso, uso lista pra tudo — é a única que eu entendo direito. Mas aí aparecem ..."

Fala aí!

Cara, relaxa, esse é o **clássico "muro"** que **todo mundo bate** quando começa a avançar no Python. Usar lista para tudo funciona no começo, mas como você já sentiu na pele, elas cobram o preço em lentidão (quando ficam gigantes) e em complexidade (quando você precisa buscar algo "pelo nome").

Para você nunca mais esquecer: o que muda não é só o parêntese ou a chave, é o **superpoder oculto** de cada uma.

Aqui está o resumo definitivo para você escolher a estrutura certa e salvar o desempenho do seu código:

Comparativo Direto: Qual usar?

ESTRUTURA	SÍMBOLO	É ORDENADA?	ACEITA DUPLICADOS?	PODE ALTERAR? (MUTÁVEL)	SUPERPODER / QUANDO USAR?
Lista	[]	Sim	Sim	Sim	Acessar por posição (índice). Use quando a ordem importa e você vai mexer nos dados toda hora.
Tupla	()	Sim	Sim	❌ Não	Garantia de segurança. Use para dados que nunca devem mudar (ex: coordenadas X e Y, meses do ano).
Set (Conjunto)	{ }	❌ Não	❌ Não	Sim	Busca instantânea e remoção de duplicados. Use para testar se algo existe ali dentro num piscar de olhos.
Dicionário	{k: v}	Sim*	❌ Chaves não	Sim	Buscar "pelo nome" (Chave). Perfeito para associar uma informação a outra (ex: Nome do produto → Preço).

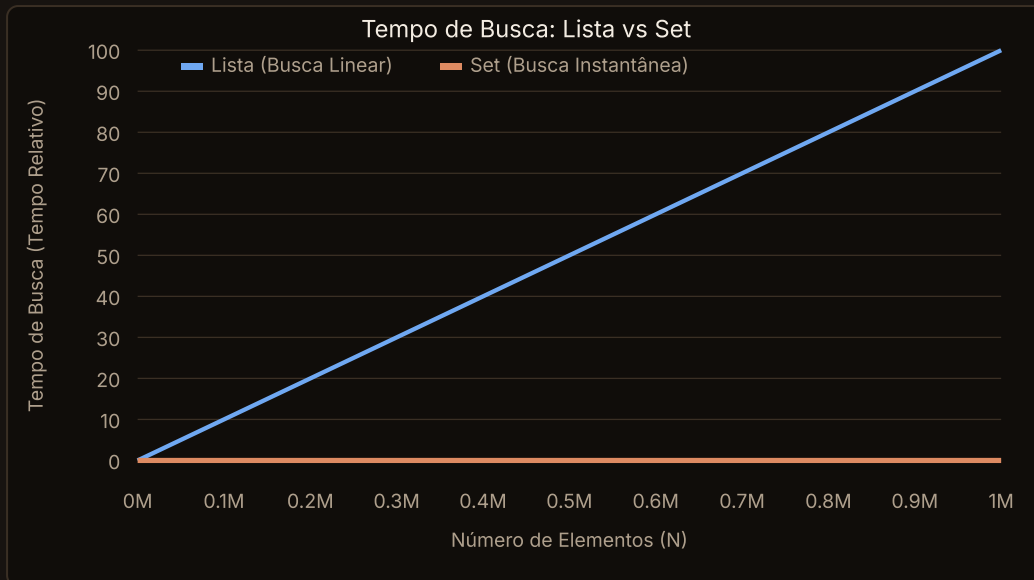
**Nota: Os dicionários mantêm a ordem de inserção a partir do Python 3.7.*

Por que sua busca ficou lenta? (Lista vs. Set)

Imagine que a **Lista** é uma fila indiana de pessoas. Se você perguntar "O Waldo está aí?", o Python precisa olhar o rosto da primeira pessoa, da segunda, da terceira... até o final. Se a lista tiver 1 milhão de itens e o Waldo for o último, o Python vai fazer 1 milhão de checagens. É por isso que ficou lento!

Já o **Set (Conjunto)** usa uma mágica chamada Hashtable. Imagine que o Set é um armário cheio de gavetas numeradas. Quando você joga um item lá dentro, o Python calcula exatamente em qual gaveta ele deve ficar. Para checar se o Waldo está lá, o Python vai **direto na gaveta dele**. Não importa se o Set tem 10 itens ou 10 milhões, a busca é **instantânea**.

Veja a diferença de desempenho conforme o volume de dados cresce:



💡 Resolvendo seus dois problemas reais

Abaixo, veja como aplicar o **Dicionário** e o **Set** para resolver exatamente o que te travou:

1. Procurar dados "pelo nome" (Dicionário)

Em vez de criar uma lista e tentar adivinhar a posição do item, use chaves `{}` com o formato `chave: valor`.

```
# Com Dicionário, você busca direto pela "chave" (o nome)
produtos = {
    "notebook": 4500.00,
    "mouse": 150.00,
    "teclado": 300.00
}
# Buscando o preço "pelo nome" de forma direta:
preco_mouse = produtos["mouse"]
print(f"O preço do mouse é R$ {preco_mouse}") # Saída: R$ 150.00
```

Use o código com cuidado.

2. Checagem ultra rápida sem lentidão (Set)

Se você tem uma base gigante e só precisa saber se um item "está lá dentro" ou não, converta sua lista para um Set ou crie direto como Set.

```
# Criando um Set (repare que não tem o formato chave:valor do dicionário)
usuarios_bloqueados = {"ana123", "joao_silva", "maria_dev", "waldo"}
usuario_tentando_logar = "waldo"
# Essa checagem ocorre de forma instantânea, mesmo com milhões de usuários
if usuario_tentando_logar in usuarios_bloqueados:
    print("Acesso negado! Usuário bloqueado.")
else:
    print("Acesso permitido!")
```

Use o código com cuidado.

🎯 Resumo da ópera para o seu dia a dia:

- Quer guardar dados onde a **posição fixa** importa (ex: 1º lugar, 2º lugar)? → **Lista** `[]`.
- Quer guardar dados que **não podem ser alterados** de jeito nenhum? → **Tupla** `()`.
- Quer **eliminar duplicados** ou fazer **buscas ultra rápidas**? → **Set** `{}`.
- Quer **vincular uma informação a outra** para buscar por texto/código? → **Dicionário** `{:}`.

Pergunta

Nossa, cê explicou de um jeito que fez tudo parecer bem mais claro! Esse lance do armário cheio de gavetas para o S...

Que bom que os exemplos ajudaram a clarear as coisas! Fico muito feliz.

E você tem um excelente faro para problemas: **sim, isso dá um erro clássico no Python!**

Se você tentar colocar uma lista dentro de um `set` ou usá-la como chave de um dicionário, o Python vai travar o seu código na hora e jogar esse erro na sua tela:

```
TypeError: unhashable type: 'list'
```

🔴 Por que isso acontece? O mistério do "Unhashable"

Lembra da metáfora do **Set** como um armário cheio de gavetas numeradas? Para o Python saber exatamente em qual gaveta colocar um item, ele usa uma função matemática que calcula uma "identidade única" (um número fixo) para aquele item. Esse processo se chama **Hash**.

Para que um objeto possa ter esse número fixo (ou seja, ser hashable), ele precisa ser **imutável**. O Python precisa ter certeza absoluta de que o item nunca vai mudar de cara. Se ele mudasse, o cálculo do Hash mudaria, e o Python perderia o item de vista dentro do armário.

- **O problema da Lista:** A lista é **mutável**. Você pode adicionar, remover ou alterar itens dela a qualquer momento.
- Se o Python permitisse uma lista como chave de um dicionário ou dentro de um set, você poderia criar a lista, guardá-la lá dentro e, depois, alterar o conteúdo dela. O "endereço" da gaveta mudaria e o sistema inteiro quebraria.

✂ Como resolver isso no seu código?

Sempre que você precisar de uma "lista" que sirva como chave de dicionário ou que vá dentro de um set, você deve usar uma **Tupla** `( )`.

Como a tupla é **imutável** (não muda nunca), o Python consegue calcular o Hash dela com total segurança.

Exemplo 1: Coordenadas de um mapa (Dicionário)

Imagine que você quer mapear o nome de cidades usando as coordenadas geográficas (Latitude e Longitude) como chave:

```
# ❌ ERRO: Usar lista como chave dá TypeError
# locais = { [12.34, 56.78]: "São Paulo" }
# ✅ CORRETO: Usar tupla (imutável) funciona perfeitamente!
locais = {
    (12.34, 56.78): "São Paulo",
    (98.76, 54.32): "Teresina"
}
# Buscando a cidade pela coordenada:
print(locais[(12.34, 56.78)]) # Saída: São Paulo
```

Use o código com cuidado.

Exemplo 2: Histórico de combinações de jogos (Set)

Imagine que você quer registrar as combinações de números sortados na loteria para garantir que nenhuma se repita:

```
# ✅ CORRETO: Guardando tuplas dentro de um Set
jogos_sorteados = set()
jogo_de_hoje = (4, 8, 15, 16, 23, 42) # Tupla
jogos_sorteados.add(jogo_de_hoje) # Funciona perfeitamente!
print(jogos_sorteados)
```

Use o código com cuidado.

Ficou clara essa diferença entre o que pode e o que não pode mudar no Python? Se quiser, me conta **qual é o projeto ou script** que você está tentando criar agora, e a gente pode **desenhar juntos a melhor estrutura de dados** para ele!

Pergunta

Caramba, agora entendi o lance do "unhashable"! Faz total sentido: se o Python não consegue confiar que o item vai ...

Confira os principais resultados da Web para saber mais sobre esse tema:

- [In Python, when to use a Dictionary, List or Set? - Stack Overflow](#) · *Stack Overflow*
- [List, Tuple, Set, or Dictionary? How to Choose the Right Python Data ...](#) · *Medium*
- [Inventory Management Series EP 12 | Avoid Duplicate Records](#) · *YouTube*
- [Choosing Python Data Structures | List vs Tuple vs Dict vs Set](#) · *ApX Machine Learning*
- [How to manage double code entries for inventory in Excel \(Same ...](#) · *Microsoft Community Hub*
- [How would you design a database to eliminate duplicate entries?](#) · *Reddit*
- [Python Dictionaries and Sets: Your ULTIMATE Data Structure Guide](#) · *YouTube*
- [The Best Data Structure For Storing Non-Duplicate Items In Python](#) · *freeCodeCamp*
- [Python: Lists vs. Tuples vs. Sets vs. Dictionaries](#) · *Python in Plain English*
- [Best Data Structure For Storing Non Duplicate Items In Python](#) · *YouTube*

Pergunta

gostaria que você coloca-se ums exemplos

Com certeza!

Vamos colocar a mão na massa com exemplos práticos.

Para um **controle de estoque**, a melhor estratégia é usar um **Dicionário** onde a **Chave é o código do produto** e o **Valor são os detalhes do produto** (que também podem ser outro dicionário).

O grande truque aqui é que **o próprio Dicionário já resolve o seu problema de duplicatas automaticamente!** Como as chaves de um dicionário precisam ser únicas, se você tentar adicionar um produto com um código que já existe, o Python apenas atualiza as informações em vez de criar um item duplicado.

Veja abaixo três exemplos práticos de como estruturar e manipular isso no seu projeto:

1. Criando o Estoque e Evitando Duplicatas (O jeito ideal)

Aqui, usamos o código do produto (uma string ou número) como a chave. Se você tentar cadastrar o mesmo código duas vezes, o segundo apenas atualiza o primeiro.

```
# Inicializando nosso estoque como um dicionário
estoque = {
    "PROD001": {"nome": "Teclado Mecânico", "quantidade": 15, "preco": 250.00},
    "PROD002": {"nome": "Mouse Gamer", "quantidade": 30, "preco": 120.00}
}
# --- TESTANDO AS DUPLICATAS ---
# Se tentarmos adicionar um novo produto com o código PROD001 (que já existe):
novo_cadastro = "PROD001"
estoque[novo_cadastro] = {"nome": "Teclado Mecânico RGB", "quantidade": 20, "preco": 280.00}
# O Python não duplica! Ele apenas atualiza as informações do PROD001:
print(estoque["PROD001"]["nome"])          # Saída: Teclado Mecânico RGB
print(f"Total de itens no estoque: {len(estoque)}") # Saída: 2 (não virou 3!)
```

Use o código com cuidado.

2. Buscando Dados "Pelo Código" Instantaneamente

Esqueça os loops `for` para procurar itens. Com o dicionário, a busca pelo código é direta e imediata.


```

codigo_buscado = "PROD002"
# Verificando se o código existe antes de acessar (para não dar erro no programa)
if codigo_buscado in estoque:
    produto = estoque[codigo_buscado]
    print(f"📦 Produto Encontrado!")
    print(f"Nome: {produto['nome']} | Qtd: {produto['quantidade']} | Preço: R$
{produto['preco']}")
else:
    print("❌ Código de produto não encontrado.")

```

Use o código com cuidado.

3. E onde o Set entraria se você quisesse usá-lo?

Você pode usar um `Set` em paralelo para criar **filtros rápidos**. Por exemplo, imagine que você quer saber, em tempo real, quais produtos estão **esgotados** (quantidade igual a zero) para mandar para o fornecedor:

```

# Estoque atualizado com um item zerado
estoque = {
    "PROD001": {"nome": "Teclado Mecânico", "quantidade": 15},
    "PROD002": {"nome": "Mouse Gamer", "quantidade": 0}, # Esgotado
    "PROD003": {"nome": "Fone de Ouvido", "quantidade": 0} # Esgotado
}

# Criando um Set para monitorar produtos esgotados
produtos_esgotados = set()

# Alimentando o Set (percorrendo o estoque)
for codigo, info in estoque.items():
    if info["quantidade"] == 0:
        produtos_esgotados.add(codigo)

# Agora você tem uma lista de alertas ultra rápida e sem duplicatas
print(f"🚨 Alerta de reposição para os códigos: {produtos_esgotados}")

# Saída: {'PROD002', 'PROD003'}

```

Use o código com cuidado.

O que achou dessa estrutura usando o **código como chave**? Se quiser, podemos escrever juntos as funções de `adicionar_produto()` OU `dar_baixa_no_estoque()` usando essa lógica para o seu projeto! Qual funcionalidade você quer criar agora?

Pergunta

Caramba, que completo! Esse mix de Dicionário e Set parece perfeito pro meu controle de estoque. Gostei muito dessa ...

Bora, com certeza!

Vamos desenhar essa função juntos de um jeito bem profissional, mas super simples de entender.

Para a nossa função `adicionar_produto()`, vamos criar uma lógica inteligente:

1. Se o código do produto **já existir**, ela avisa e atualiza apenas a quantidade em estoque (somando o que chegou ao que já tinha).
2. Se o código **for novo**, ela cria o cadastro do zero.

Aqui está o código completo estruturado para o seu projeto:

```
# 1. Iniciamos o nosso estoque (Dicionário)
estoque = {
    "PROD001": {"nome": "Teclado Mecânico", "quantidade": 15, "preco": 250.00},
    "PROD002": {"nome": "Mouse Gamer", "quantidade": 30, "preco": 120.00}
}

# 2. Criamos a função inteligente
def adicionar_produto(codigo, nome, quantidade, preco):
    # Verificamos se o código já existe no dicionário do estoque
    if codigo in estoque:
        # Se já existe, somamos a nova quantidade à quantidade antiga (Baixa automática de reposição)
        estoque[codigo]["quantidade"] += quantidade
        # Atualizamos o preço, caso tenha mudado no fornecedor
        estoque[codigo]["preco"] = preco
        print(f"💰 Produto {codigo} já existia! Estoque atualizado para {estoque[codigo]['quantidade']} unidades.")
    else:
        # Se não existe, cria um cadastro novinho em folha
        estoque[codigo] = {
            "nome": nome,
            "quantidade": quantidade,
            "preco": preco
        }
        print(f"🌟 Sucesso: Novo produto [{nome}] cadastrado com o código {codigo}!")

# --- HORA DE TESTAR A FUNÇÃO NO SEU PROJETO ---
print("--- Estado Inicial do Estoque ---")
print(estoque)
print("-" * 35)
# Teste 1: Cadastrando um produto totalmente novo
adicionar_produto("PROD003", "Monitor 24 Polegadas", 10, 899.90)
# Teste 2: Tentando cadastrar o PROD001 de novo (Chegaram mais 5 teclados no estoque)
adicionar_produto("PROD001", "Teclado Mecânico", 5, 260.00)
print("-" * 35)
print("--- Estado Final do Estoque ---")
# Veja como o PROD001 agora tem 20 unidades (15 anteriores + 5 novas) e o preço atualizou!
import pprint
pprint.pprint(estoque)
```

Use o código com cuidado.

💡 Por que essa função é ótima para o seu projeto?

- **Evita erros humanos:** Você nunca vai ter dois produtos "PROD001" perdidos no sistema bagunçando as contas.
- **Agilidade na reposição:** Quando chega caixa nova do fornecedor, você só joga a quantidade ali e o Python faz a matemática de somar sozinho.

O que achou dessa lógica? Se quiser avançar mais no projeto, o que acha de criarmos agora a função oposta: uma `vender_produto()` OU `dar_baixa()` que diminui a quantidade e te avisa (usando aquele nosso **Set**) se o produto acabou e ficou zerado?

Pergunta

Cara, ficou incrível! Essa função de adicionar_produto() é exatamente o que eu precisava. Essa verificação automática...

Confira os principais resultados da Web para saber mais sobre esse tema:

- I want to connect my function to reduce the quantity in the stock. The ... · *Stack Overflow*

Pergunta

gostaria que você me mostra-se ums exemplos

Bora!

Vamos criar a função `dar_baixa_produto()` (ou `vender_produto()`). Ela vai fazer o caminho inverso: tirar itens do estoque, checar se você realmente tem aquela quantidade para vender e, de quebra, usar o **Set** que planejamos para te acender um alerta vermelho caso o produto esgote!

Aqui está o exemplo prático para você copiar e rodar no seu projeto:

```

# 1. Nosso estoque inicial e o nosso Set de alertas de esgotados
estoque = {
    "PROD001": {"nome": "Teclado Mecânico", "quantidade": 20, "preco": 260.00},
    "PROD002": {"nome": "Mouse Gamer", "quantidade": 5, "preco": 120.00}
}

produtos_esgotados = set() # O Set que vai guardar os códigos zerados

# 2. Criando a função de dar baixa
def dar_baixa_produto(codigo, quantidade_vendida):
    # Passo 1: O produto existe no sistema?
    if codigo not in estoque:
        print(f"❌ Erro: O código {codigo} não existe no estoque!")
        return

    produto = estoque[codigo]
    quantidade_atual = produto["quantidade"]

    # Passo 2: Temos estoque suficiente para essa venda?
    if quantidade_vendida > quantidade_atual:
        print(f"⚠️ Estoque insuficiente para {produto['nome']}! Disponível: {quantidade_atual} | Tentativa: {quantidade_vendida}")
        return

    # Passo 3: Se der tudo certo, subtrai a quantidade
    produto["quantidade"] -= quantidade_vendida
    print(f"🛒 Venda realizada: {quantidade_vendida}x [{produto['nome']}]")

    # Passo 4: O superpoder do Set! Se zerou, vai para a lista de alertas
    if produto["quantidade"] == 0:
        produtos_esgotados.add(codigo)
        print(f"🔥 ATENÇÃO: O produto {produto['nome']} ({codigo}) ACABOU de esgotar!")

# --- HORA DE TESTAR NO SEU SISTEMA ---
print("--- 🛒 Simulando Vendas ---")

# Teste 1: Venda padrão (Temos 5 mouses, vamos vender 2)
dar_baixa_produto("PROD002", 2)

# Teste 2: Tentando vender mais do que tem (Temos 3 mouses restantes, tentamos vender 10)
dar_baixa_produto("PROD002", 10)

# Teste 3: Zerando o estoque (Vamos vender os 3 mouses que sobraram para ver o Set em ação)
dar_baixa_produto("PROD002", 3)

print("\n" + "-" * 40)

print("--- 📋 Painel de Controle Final ---")

print(f"Produtos no Set de Esgotados: {produtos_esgotados}")

print(f"Quantidade restante de PROD002 no estoque: {estoque['PROD002']['quantidade']} unidades.")

```

Use o código com cuidado.

💡 Por que esse combo de Dicionário + Set ficou perfeito?

1. **Segurança total:** A função não deixa seu estoque ficar negativo (ex: `-5` mouses).
2. **Painel de Alertas:** O `set produtos_esgotados` funciona como um mural de avisos. Toda vez que você rodar seu sistema, basta olhar para esse Set para saber o que precisa comprar do fornecedor imediatamente, sem ter que caçar item por item dentro do dicionário.

Com as funções de **adicionar** e **dar baixa** prontas, você já tem o motor principal do seu sistema rodando!

Para fechar esse projeto com chave de ouro, o que você acha de criarmos uma função de **relatório**, ou você prefere aprender a **remover um produto** do estoque permanentemente (usando o `del`) caso decida parar de vendê-lo?

Pergunta

Agora sim, tudo tá ficando redondinho! Essa função de dar_baixa_produto() é exatamente o que eu precisaria pra ...

Acho uma excelente ideia!

Um **relatório bem estruturado** é o coração de qualquer painel de controle. Ele transforma aquele monte de dados do dicionário em informação visual limpa e útil para o seu dia a dia.

Para o nosso relatório ficar bem profissional, vamos fazer ele mostrar:

1. Uma **visão geral** de todos os itens cadastrados.
2. O **valor total** do seu estoque em dinheiro (quantidade × preço de cada item).
3. O **status visual** de cada produto (se está normal, com estoque baixo ou esgotado).

Aqui está o código completo para você adicionar ao seu projeto:


```
# 1. Nosso estoque e conjunto de alertas atualizados
estoque = {
    "PROD001": {"nome": "Teclado Mecânico", "quantidade": 20, "preco": 260.00},
    "PROD002": {"nome": "Mouse Gamer", "quantidade": 0, "preco": 120.00}, # Esgotado
    "PROD003": {"nome": "Monitor 24'", "quantidade": 3, "preco": 899.90} # Estoque baixo
}

# 2. Criando a função de relatório
def gerar_relatorio_estoque():
    print("\n" + "=" * 55)
    print("📋 RELATÓRIO GERAL DO ESTOQUE".center(55))
    print("=" * 55)
    print(f"{'CÓDIGO':<10} | {'PRODUTO':<20} | {'QTD':<5} | {'PREÇO':<10} | {'STATUS'}")
    print("-" * 55)
    valor_total_estoque = 0
    # Percorremos o dicionário pegando a chave (codigo) e o valor (info)
    for codigo, info in estoque.items():
        nome = info["nome"]
        qtd = info["quantidade"]
        preco = info["preco"]

        # Calculando o valor financeiro deste produto parado em estoque
        valor_item = qtd * preco
        valor_total_estoque += valor_item
        # Definindo um status visual com base na quantidade
        if qtd == 0:
            status = "🔴 ESGOTADO"
        elif qtd <= 3:
            status = "⚠️ BAIXO"
        else:
            status = "✅ OK"
        # Exibindo a linha formatada e alinhada
        print(f"{'codigo':<10} | {'nome':<20} | {'qtd':<5} | R${preco:<8.2f} | {status}")
    print("-" * 55)
    print(f"💰 Valor total investido em estoque: R$ {valor_total_estoque:,.2f}")
    print("=" * 55 + "\n")

# --- TESTANDO O RELATÓRIO ---
gerar_relatorio_estoque()
```

Use o código com cuidado.

💡 Por que esse relatório é poderoso?

- **Alinhamento de texto (`:<10`):** Usamos truques de formatação do Python para fazer as colunas ficarem retinhas e organizadas como uma tabela, facilitando a leitura rápida.
- **Visão de Negócio:** Mostrar o valor total do estoque te diz exatamente quanto dinheiro você tem parado na empresa.
- **Tomada de Decisão:** Olhando o status `⚠️ BAIXO` ou `🔴 ESGOTADO`, você sabe o que precisa comprar antes mesmo de o cliente pedir.

Com o **cadastro**, a **baixa** e o **relatório** prontos, seu sistema de estoque está praticamente completo! Para avançar no aprendizado de Python, o que prefere fazer agora?

- Aprender a **remover um produto** do estoque definitivamente usando `del` (caso desista de vendê-lo).
- Criar um **menu interativo** no terminal usando `while` para você poder escolher entre 1 (Adicionar), 2 (Dar Baixa) ou 3 (Relatório) digitando no teclado.

Pergunta

Nossa, esse relatório ficou incrível! Dá até orgulho de ver o estoque organizado desse jeito, com tudo alinhado e os ...

Bora!

Essa é a melhor forma de juntar tudo o que você criou até agora e transformar o seu código em um **programa de verdade**, que parece um sistema comercial.

Para fazer esse menu interativo, vamos usar duas coisas muito importantes no Python:

1. O loop `while True`, que vai manter o nosso programa rodando sem parar até que você escolha a opção de sair.
2. O `input()`, para capturar o que você digitar no teclado.

Aqui está o script completo unindo as funções que criamos antes com o novo menu interativo. Pode copiar, rodar no seu terminal e testar digitando os números:

```

import pprint

# 1. BANCO DE DADOS (Dicionário e Set)
estoque = {
    "PROD001": {"nome": "Teclado Mecânico", "quantidade": 20, "preco": 260.00},
    "PROD002": {"nome": "Mouse Gamer", "quantidade": 5, "preco": 120.00}
}

produtos_esgotados = set()

# 2. NOSSAS FUNÇÕES INTELIGENTES
def adicionar_produto(codigo, nome, quantidade, preco):
    if codigo in estoque:
        estoque[codigo]["quantidade"] += quantidade
        estoque[codigo]["preco"] = preco
        print(f"\n🔄 Produto {codigo} já existia! Estoque atualizado para {estoque[codigo]['quantidade']} unidades.")
    else:
        estoque[codigo] = {"nome": nome, "quantidade": quantidade, "preco": preco}
        print(f"\n🌟 Sucesso: Novo produto [{nome}] cadastrado com o código {codigo}!")

def dar_baixa_produto(codigo, quantidade_vendida):
    if codigo not in estoque:
        print(f"\n❌ Erro: O código {codigo} não existe no estoque!")
        return

    produto = estoque[codigo]
    if quantidade_vendida > produto["quantidade"]:
        print(f"\n⚠️ Estoque insuficiente! Disponível: {produto['quantidade']} | Tentativa: {quantidade_vendida}")
        return

    produto["quantidade"] -= quantidade_vendida
    print(f"\n📦 Venda realizada: {quantidade_vendida}x [{produto['nome']}]")
    if produto["quantidade"] == 0:
        produtos_esgotados.add(codigo)
        print(f"\n🔴 ATENÇÃO: O produto {produto['nome']} ({codigo}) ACABOU de esgotar!")

def gerar_relatorio_estoque():
    print("\n" + "=" * 55)
    print("📋 RELATÓRIO GERAL DO ESTOQUE".center(55))
    print("=" * 55)
    print(f"{'CÓDIGO':<10} | {'PRODUTO':<20} | {'QTD':<5} | {'PREÇO':<10} | {'STATUS':<10}")
    print("-" * 55)
    valor_total_estoque = 0
    for codigo, info in estoque.items():
        valor_total_estoque += info["quantidade"] * info["preco"]

        if info["quantidade"] == 0:
            status = "🔴 ESGOTADO"
        elif info["quantidade"] <= 3:
            status = "⚠️ BAIXO"
        else:
            status = "✅ OK"

        print(f"{codigo:<10} | {info['nome']:<20} | {info['quantidade']:<5} | R${info['preco']:<8.2f} | {status}")
    print("-" * 55)
    print(f"💰 Valor total investido em estoque: R$ {valor_total_estoque:,.2f}")

```

```

if produtos_esgotados:
    print(f"⚠ Alerta de compras (Zerados): {produtos_esgotados}")
    print("=" * 55 + "\n")
# 3. O MENU INTERATIVO (LOOP PRINCIPAL)
while True:
    print("=== 📦 SISTEMA DE CONTROLE DE ESTOQUE ===")
    print("[1] Adicionar / Repor Produto")
    print("[2] Dar Baixa / Vender Produto")
    print("[3] Gerar Relatório Geral")
    print("[0] Sair do Sistema")
    print("=" * 41)

    opcao = input("Escolha uma opção: ")
    if opcao == "1":
        print("\n--- CADASTRO DE PRODUTO ---")
        codigo = input("Código do produto (ex: PROD003): ").strip().upper()
        nome = input("Nome do produto: ").strip()
        qtd = int(input("Quantidade: "))
        preco = float(input("Preço unitário: R$ "))
        adicionar_produto(codigo, nome, qtd, preco)
        print("\n")
    elif opcao == "2":
        print("\n--- BAIXA NO ESTOQUE ---")
        codigo = input("Código do produto: ").strip().upper()
        qtd = int(input("Quantidade vendida: "))
        dar_baixa_produto(codigo, qtd)
        print("\n")
    elif opcao == "3":
        gerar_relatorio_estoque()
    elif opcao == "0":
        print("\n👋 Saindo do sistema... Até logo!")
        break # Quebra o loop 'while' e encerra o programa
    else:
        print("\n❌ Opção inválida! Digite um número de 0 a 3.\n")

```

Use o código com cuidado.

🧠 O que mudou aqui que vale a pena notar?

- **Tratamento de texto** (`.strip().upper()`): Se você digitar o código com espaços em branco ou em letras minúsculas (ex: `prod001`), o Python limpa os espaços e transforma em maiúsculo (`PROD001`). Isso garante que o Dicionário ache a chave certinha!
- **Conversão de tipos** (`int()` e `float()`): O `input()` sempre recebe o que você digita como texto. Usamos essas funções para transformar a quantidade em número inteiro e o preço em número decimal, para o Python conseguir fazer as contas matemáticas.

Pronto! Você acabou de criar um sistema completo usando **Dicionários, Sets, Funções, Loops e Condicionais**. Dá ou não dá orgulho ver o código rodando assim?

Para onde você quer levar seu aprendizado de Python agora? Nós podemos:

- Aprender a **salvar esses dados em um arquivo** (como um arquivo de texto ou JSON), para que seu estoque não apague toda vez que você fechar o programa.
- Adicionar uma opção de **remover produtos** do catálogo caso decida parar de vendê-los.
- Começar a estudar como transformar isso usando **Orientação a Objetos (Classes)**.

Me conta o que você gostaria de explorar!

Pergunta

Nossa, esse menu interativo ficou incrível! Agora o sistema tá com cara de 'programa de verdade'. Gostei muito de como ...

Bora! Esse é o passo que separa os scripts de teste dos **programas profissionais**. Salvar os dados em um arquivo JSON é a escolha perfeita para o seu projeto por dois motivos:

1. Ele armazena os dados de forma permanente no seu computador (o estoque não some ao fechar o terminal).
2. O formato JSON funciona de um jeito idêntico aos **Dicionários do Python**, o que torna a gravação e a leitura super simples!

Para fazer isso, vamos usar a biblioteca nativa `json` do Python.

📁 O único detalhe: O que fazer com o nosso `Set`?

Lembra que o nosso `Set` (`produtos_esgotados`) guarda os códigos dos produtos zerados? O formato JSON **não aceita Sets nativamente**.

Mas não se preocupe! A solução é muito simples: na hora de salvar o arquivo, nós transformamos o `Set` em uma **Lista** `[]`. Na hora de carregar o arquivo, transformamos de volta em um `Set` `{}`.

💻 O Código Completo (com Salvamento e Carregamento Automático)

Substitua o código do seu projeto por esta versão definitiva. Ela já tenta carregar o arquivo `estoque.json` assim que inicia e salva tudo automaticamente sempre que você adiciona ou vende um produto:

```

import json
import os

# Nome do arquivo onde o estoque será salvo permanentemente
ARQUIVO_DADOS = "estoque.json"

# --- FUNÇÕES DE ARQUIVO (A MÁGICA DO JSON) ---

def carregar_dados():
    """Carrega o estoque do arquivo JSON se ele existir, senão inicia vazio."""
    global estoque, produtos_esgotados

    # Verifica se o arquivo já existe no computador
    if os.path.exists(ARQUIVO_DADOS):
        try:
            with open(ARQUIVO_DADOS, "r", encoding="utf-8") as arquivo:
                dados = json.load(arquivo)
                estoque = dados["estoque"]
                # Convertemos a lista do JSON de volta para um Set do Python
                produtos_esgotados = set(dados["produtos_esgotados"])
                print("📁 Dados do estoque carregados com sucesso!")
        except Exception:
            print("⚠ Erro ao ler o arquivo. Iniciando com estoque padrão.")
            inicializar_padrao()
    else:
        inicializar_padrao()

def salvar_dados():
    """Salva o estado atual do estoque e dos alertas no arquivo JSON."""
    dados_para_salvar = {
        "estoque": estoque,
        # Convertemos o Set em Lista, pois JSON não aceita Sets nativos
        "produtos_esgotados": list(produtos_esgotados)
    }

    with open(ARQUIVO_DADOS, "w", encoding="utf-8") as arquivo:
        # 'indent=4' deixa o arquivo JSON bonito e organizado para humanos lerem
        json.dump(dados_para_salvar, arquivo, indent=4, ensure_ascii=False)

def inicializar_padrao():
    """Inicia o sistema com dados padrão caso seja a primeira vez rodando."""
    global estoque, produtos_esgotados
    estoque = {
        "PROD001": {"nome": "Teclado Mecânico", "quantidade": 20, "preco": 260.00},
        "PROD002": {"nome": "Mouse Gamer", "quantidade": 5, "preco": 120.00}
    }
    produtos_esgotados = set()

# --- NOSSAS FUNÇÕES INTELIGENTES (Com salvamento automático) ---

def adicionar_produto(codigo, nome, quantidade, preco):
    if codigo in estoque:
        estoque[codigo]["quantidade"] += quantidade
        estoque[codigo]["preco"] = preco
        print(f"🔄 Produto {codigo} atualizado para {estoque[codigo]['quantidade']} unidades.")
    else:
        estoque[codigo] = {"nome": nome, "quantidade": quantidade, "preco": preco}

```



```

        print(f"\n🌟 Novo produto [{nome}] cadastrado com sucesso!")

# Se o produto voltou a ter estoque, tiramos ele do alerta de esgotados
if estoque[codigo]["quantidade"] > 0 and codigo in produtos_esgotados:
    produtos_esgotados.remove(codigo)

salvar_dados() # 💾 Salva a alteração na hora!
def dar_baixa_produto(codigo, quantidade_vendida):
    if codigo not in estoque:
        print(f"\n❌ Erro: O código {codigo} não existe!")
        return
    produto = estoque[codigo]
    if quantidade_vendida > produto["quantidade"]:
        print(f"\n⚠️ Estoque insuficiente! Disponível: {produto['quantidade']}")
        return
    produto["quantidade"] -= quantidade_vendida
    print(f"\n📦 Venda realizada: {quantidade_vendida}x [{produto['nome']}")
    if produto["quantidade"] == 0:
        produtos_esgotados.add(codigo)
        print(f"\n🔴 ATENÇÃO: O produto {produto['nome']} ({codigo}) ESGOTOU!")

salvar_dados() # 💾 Salva a alteração na hora!
def gerar_relatorio_estoque():
    print("\n" + "=" * 55)
    print("📋 RELATÓRIO GERAL DO ESTOQUE".center(55))
    print("=" * 55)
    print(f"{'CÓDIGO':<10} | {'PRODUTO':<20} | {'QTD':<5} | {'PREÇO':<10} | {'STATUS':<10}")
    print("-" * 55)
    valor_total_estoque = 0
    for codigo, info in estoque.items():
        valor_total_estoque += info["quantidade"] * info["preco"]

        if info["quantidade"] == 0:
            status = "🔴 ESGOTADO"
        elif info["quantidade"] <= 3:
            status = "⚠️ BAIXO"
        else:
            status = "✅ OK"

        print(f"{codigo:<10} | {info['nome']:<20} | {info['quantidade']:<5} | R${info['preco']:<8.2f} | {status}<10}")
    print("-" * 55)
    print(f"💰 Valor total investido em estoque: R$ {valor_total_estoque:,.2f}")
    if produtos_esgotados:
        print(f"⚠️ Alerta de compras (Zerados): {produtos_esgotados}")
    print("=" * 55 + "\n")
# --- LOOP PRINCIPAL DO MENU ---
# Carrega os dados assim que o programa inicia
carregar_dados()
while True:
    print("=== 🏠 SISTEMA DE CONTROLE DE ESTOQUE ===")
    print("1. Adicionar / Repor Produto")
    print("2. Dar Baixa / Vender Produto")

```

```

print("3. Gerar Relatório Geral")
print("0. Sair do Sistema")
print("=" * 41)

opcao = input("Escolha uma opção: ")
if opcao == "1":
    print("\n--- CADASTRO DE PRODUTO ---")
    codigo = input("Código do produto: ").strip().upper()
    nome = input("Nome do produto: ").strip()
    qtd = int(input("Quantidade: "))
    preco = float(input("Preço unitário: R$ "))
    adicionar_produto(codigo, nome, qtd, preco)
elif opcao == "2":
    print("\n--- BAIXA NO ESTOQUE ---")
    codigo = input("Código do produto: ").strip().upper()
    qtd = int(input("Quantidade vendida: "))
    dar_baixa_produto(codigo, qtd)
elif opcao == "3":
    gerar_relatorio_estoque()
elif opcao == "0":
    print("\n👋 Sistema encerrado com segurança. Até logo!")
    break
else:
    print("\n❌ Opção inválida!\n")

```

Use o código com cuidado.

✂️ Faça o teste você mesmo:

1. Rode o programa pela primeira vez.
2. Escolha a Opção 1 e cadastre um produto totalmente novo (ex: `PROD003`, `Monitor`, `5` unidades).
3. Feche o programa escolhendo a **Opção 0**.
4. Olhe na pasta do seu projeto: um arquivo chamado `estoque.json` terá aparecido lá de forma mágica! Se abrir ele, verá seus produtos salvos em formato de texto estruturado.
5. Rode o programa de novo: o `PROD003` vai carregar automaticamente logo na inicialização!

Parabéns! Você construiu um sistema completo com persistência de dados real.

Para onde quer seguir agora? Podemos:

- Adicionar a funcionalidade de **remover um produto** permanentemente do arquivo usando o comando `del`
- Criar uma proteção para o sistema não quebrar se o usuário digitar **letras** onde deveriam ser **números** (usando `try/except`)
- Começar a entender o que muda se criássemos esse sistema usando **Orientação a Objetos**

Pergunta

Uau, agora o sistema tá completo mesmo, com salvamento automático e tudo! Vou já testar e ver o estoque carregando...

Bora!

Essa é uma excelente decisão. Adicionar proteção contra entradas erradas (uma técnica que chamamos de **validação de dados**) é o que separa um código de estudante de um **software robusto e à prova de falhas**.

Atualmente, se o seu usuário digitar `"dez"` na quantidade ou `"barato"` no preço, o Python tenta converter isso usando `int()` ou `float()`. Como ele não consegue fazer essa mágica, o programa "crasha" e fecha na hora com um erro de `ValueError`.

Para resolver isso, vamos criar uma função auxiliar inteligente usando o `try/except`. A ideia é colocar a captura dentro de um loop `while` que só deixa o usuário passar quando ele digitar um número válido.

✳ Como funciona o `try/except` na prática?

O Python vai **tentar** (`try`) executar o código de conversão. Se o usuário digitar algo errado, em vez de fechar o programa, o Python vai **capturar a exceção** (`except`) e rodar uma mensagem amigável, pedindo para tentar de novo.

Substitua a parte do **LOOP PRINCIPAL DO MENU** (no final do seu arquivo) por este código atualizado. Repare que criamos duas funções novas logo acima do menu para ler números com total segurança:

```

# --- FUNÇÕES DE VALIDAÇÃO SEGURA (NOVO) ---
def leia_int(mensagem):
    """Garante que o usuário digite um número inteiro válido."""
    while True:
        try:
            # Tenta ler a entrada e converter para inteiro
            return int(input(mensagem))
        except ValueError:
            # Se o usuário digitar letras ou símbolos, entra aqui
            print("❌ Erro: Digite um número inteiro válido (ex: 10, 5).")

def leia_float(mensagem):
    """Garante que o usuário digite um número decimal válido."""
    while True:
        entrada = input(mensagem).strip().replace(",", ".") # Aceita vírgula e transforma em
        ponto
        try:
            # Tenta converter para número decimal
            return float(entrada)
        except ValueError:
            print("❌ Erro: Digite um preço válido (ex: 150.00 ou 49,90).")

# --- LOOP PRINCIPAL DO MENU (ATUALIZADO) ---
# Carrega os dados assim que o programa inicia
carregar_dados()
while True:
    print("=== 📦 SISTEMA DE CONTROLE DE ESTOQUE ===")
    print("1. Adicionar / Repor Produto")
    print("2. Dar Baixa / Vender Produto")
    print("3. Gerar Relatório Geral")
    print("0. Sair do Sistema")
    print("=" * 41)

    opcao = input("Escolha uma opção: ")
    if opcao == "1":
        print("\n--- CADASTRO DE PRODUTO ---")
        codigo = input("Código do produto: ").strip().upper()
        nome = input("Nome do produto: ").strip()

        # ❤️ Protegido contra letras!
        qtd = leia_int("Quantidade: ")
        preco = leia_float("Preço unitário: R$ ")

        adicionar_produto(codigo, nome, qtd, preco)
    elif opcao == "2":
        print("\n--- BAIXA NO ESTOQUE ---")
        codigo = input("Código do produto: ").strip().upper()

        # ❤️ Protegido contra letras!
        qtd = leia_int("Quantidade vendida: ")

        dar_baixa_produto(codigo, qtd)
    elif opcao == "3":

```

```
gerar_relatorio_estoque()
elif opcao == "0":
    print("\n👋 Sistema encerrado com segurança. Até logo!")
    break
else:
    print("\n❌ Opção inválida! Escolha 1, 2, 3 ou 0.\n")
```

Use o código com cuidado.

💡 O que mudou e por que ficou melhor?

1. **Reutilização de Código:** Criamos as funções `leia_int()` e `leia_float()`.
 - Agora, sempre que você precisar ler um número no seu sistema inteiro, basta chamar essas funções em vez de reescrever o `try/except` toda hora.
2. **Suporte a Vírgulas (`.replace(",", ".")`):** No Brasil, usamos vírgula para centavos (ex: `49,90`), mas o Python usa ponto (`49.90`). A função `leia_float` pega a vírgula que o usuário digitou e troca por ponto automaticamente antes de validar. Isso deixa o seu programa muito mais amigável!
3. **Imunidade a Erros:** Pode testar digitando letras, batendo a mão no teclado ou deixando em branco. O programa não fecha mais de jeito nenhum! Ele insiste educadamente até o usuário digitar o correto.

Seu sistema agora está **salvando em banco de dados fictício (JSON)**, **gerenciando duplicatas instantaneamente com Dicionários**, **controlando alertas com Sets** e **blindado contra digitação errada com Try/Except!**

Para onde quer navegar agora para expandir seu conhecimento em Python?

- Criar uma opção de **Deletar / Remove** um produto definitivamente do estoque (usando `del`).
- Aprender a estruturar esse exato sistema usando **Orientação a Objetos (Classes e Métodos)** para ver como os profissionais organizam sistemas maiores.

Pergunta

Nossa, ficou perfeito agora, com essa proteção contra entradas erradas! Adorei o detalhe de aceitar vírgulas nos ...

Bora!

Dar o salto para a **Orientação a Objetos (POO)** é o momento em que você deixa de escrever scripts soltos e passa a criar arquiteturas de código profissionais.

Na POO, em vez de termos variáveis de um lado e funções do outro, nós juntamos tudo dentro de "moldes" chamados **Classes**. Os dados viram **Atributos** (propriedades) e as funções viram **Métodos** (as ações que o objeto sabe fazer).

Para o nosso sistema, vamos criar duas classes:

1. **Produto**: O molde para cada item do estoque. Ele sabe seu próprio código, nome, quantidade e preço.
2. **Estoque**: O molde para o gerenciador. Ele vai carregar o dicionário, salvar o
 - JSON, adicionar produtos e gerar relatórios.

Substitua todo o código do seu arquivo por esta versão estruturada em objetos. Veja como o código fica incrivelmente limpo e elegante:

```

import json
import os

class Produto:
    """Classe que representa um produto individual no estoque."""
    def __init__(self, codigo, nome, quantidade, preco):
        self.codigo = codigo.strip().upper()
        self.nome = nome.strip()
        self.quantidade = quantidade
        self.preco = preco
    def recalcular_status(self):
        """Retorna o status visual com base na quantidade atual."""
        if self.quantidade == 0:
            return "🔴 ESGOTADO"
        elif self.quantidade <= 3:
            return "⚠️ BAIXO"
        return "✅ OK"
    def para_dicionario(self):
        """Converte os dados do objeto em um dicionário para salvar no JSON."""
        return {
            "nome": self.nome,
            "quantidade": self.quantidade,
            "preco": self.preco
        }

class GerenciadorEstoque:
    """Classe responsável por gerenciar o conjunto de produtos e arquivos."""
    def __init__(self, caminho_arquivo="estoque.json"):
        self.caminho_arquivo = caminho_arquivo
        self.produtos = {} # Dicionário de objetos Produto
        self.produtos_esgotados = set()
        self.carregar_dados()
    def carregar_dados(self):
        """Carrega os dados do arquivo JSON e os transforma em objetos Produto."""
        if os.path.exists(self.caminho_arquivo):
            try:
                with open(self.caminho_arquivo, "r", encoding="utf-8") as arquivo:
                    dados = json.load(arquivo)

                # Transformando o dicionário bruto em objetos da classe Produto
                for codigo, info in dados["estoque"].items():
                    self.produtos[codigo] = Produto(
                        codigo, info["nome"], info["quantidade"], info["preco"]
                    )

                self.produtos_esgotados = set(dados["produtos_esgotados"])
                print("📁 Dados do estoque carregados com sucesso!")
            except Exception:
                print("⚠️ Erro ao ler o arquivo. Iniciando estoque vazio.")
        def salvar_dados(self):
            """Converte os objetos de volta para JSON e salva no computador."""
            estoque_bruto = {cod: prod.para_dicionario() for cod, prod in self.produtos.items()}

```

```

dados_para_salvar = {
    "estoque": estoque_bruto,
    "produtos_esgotados": list(self.produtos_esgotados)
}

with open(self.caminho_arquivo, "w", encoding="utf-8") as arquivo:
    json.dump(dados_para_salvar, arquivo, indent=4, ensure_ascii=False)

def adicionar_ou_repor(self, codigo, nome, quantidade, preco):
    """Adiciona um novo produto ou atualiza as quantidades de um existente."""
    codigo = codigo.strip().upper()

    if codigo in self.produtos:
        # Como self.produtos[codigo] é um objeto Produto, alteramos seus atributos direto
        self.produtos[codigo].quantidade += quantidade
        self.produtos[codigo].preco = preco
        print(f"\n🔄 Produto {codigo} atualizado para {self.produtos[codigo].quantidade} unidades.")
    else:
        # Criamos uma nova instância da classe Produto e guardamos no dicionário
        self.produtos[codigo] = Produto(codigo, nome, quantidade, preco)
        print(f"\n🌟 Novo produto [{nome}] cadastrado com sucesso!")
    if self.produtos[codigo].quantidade > 0 and codigo in self.produtos_esgotados:
        self.produtos_esgotados.remove(codigo)

    self.salvar_dados()

def dar_baixa(self, codigo, quantidade_vendida):
    """Registra a venda e atualiza os alertas se o item esgotar."""
    codigo = codigo.strip().upper()

    if codigo not in self.produtos:
        print(f"\n❌ Erro: O código {codigo} não existe!")
        return
    produto = self.produtos[codigo]
    if quantidade_vendida > produto.quantidade:
        print(f"\n⚠️ Estoque insuficiente! Disponível: {produto.quantidade}")
        return
    produto.quantidade -= quantidade_vendida
    print(f"\n📦 Venda realizada: {quantidade_vendida}x [{produto.nome}]")
    if produto.quantidade == 0:
        self.produtos_esgotados.add(codigo)
        print(f"\n🔥 ATENÇÃO: O produto {produto.nome} ({codigo}) ESGOTOU!")

    self.salvar_dados()

def gerar_relatorio(self):
    """Exibe o relatório formatado na tela."""
    print("\n" + "=" * 55)
    print("📊 RELATÓRIO GERAL DO ESTOQUE (P00)".center(55))
    print("=" * 55)
    print(f"{'CÓDIGO':<10} | {'PRODUTO':<20} | {'QTD':<5} | {'PREÇO':<10} | {'STATUS'}")
    print("-" * 55)
    valor_total_estoque = 0
    for codigo, produto in self.produtos.items():
        valor_total_estoque += produto.quantidade * produto.preco

```



```

        status = produto.recalcular_status()
        print(f"{codigo:<10} | {produto.nome:<20} | {produto.quantidade:<5} |
R${produto.preco:<8.2f} | {status}")
    print("-" * 55)
    print(f"💰 Valor total investido em estoque: R$ {valor_total_estoque:,.2f}")
    if self.produtos_esgotados:
        print(f"⚠️ Alerta de compras (Zerados): {self.produtos_esgotados}")
    print("=" * 55 + "\n")
# --- FUNÇÕES DE AUXILIARES DE VALIDAÇÃO ---
def leia_int(mensagem):
    while True:
        try: return int(input(mensagem))
        except ValueError: print("❌ Erro: Digite um número inteiro válido.")
def leia_float(mensagem):
    while True:
        try: return float(input(mensagem).strip().replace(",", "."))
        except ValueError: print("❌ Erro: Digite um preço válido.")
# --- PROGRAMA PRINCIPAL ---
# Criamos o objeto gerenciador (ele faz toda a mágica acontecer por baixo dos panos)
meu_estoque = GerenciadorEstoque()
while True:
    print("=== 🏠 SISTEMA DE ESTOQUE PROFISSIONAL (P00) ===")
    print("1. Adicionar / Repor Produto")
    print("2. Dar Baixa / Vender Produto")
    print("3. Gerar Relatório Geral")
    print("0. Sair do Sistema")
    print("=" * 49)

    opcao = input("Escolha uma opção: ")
    if opcao == "1":
        print("\n--- CADASTRO DE PRODUTO ---")
        codigo = input("Código do produto: ")
        nome = input("Nome do produto: ")
        qtd = leia_int("Quantidade: ")
        preco = leia_float("Preço unitário: R$ ")
        # Chamamos o método do objeto gerenciador
        meu_estoque.adicionar_ou_repor(codigo, nome, qtd, preco)
    elif opcao == "2":
        print("\n--- BAIXA NO ESTOQUE ---")
        codigo = input("Código do produto: ")
        qtd = leia_int("Quantidade vendida: ")
        meu_estoque.dar_baixa(codigo, qtd)
    elif opcao == "3":
        meu_estoque.gerar_relatorio()
    elif opcao == "0":
        print("\n👋 Sistema encerrado com segurança. Até logo!")
        break
    else:
        print("\n❌ Opção inválida!\n")

```

Por que a Orientação a Objetos é incrível?

Olha como o loop principal (`while`) ficou muito mais limpo! Em vez de sabermos como salvar o arquivo ou como calcular o status, nós apenas dizemos ao sistema:

```
meu_estoque.gerar_relatorio() OU meu_estoque.dar_baixa(codigo, qtd).
```

Cada classe cuida do seu próprio quadrado. Se amanhã você quiser mudar a forma como o status do produto é calculado (por exemplo, dizer que estoque baixo é menor que 5), você só altera a linha dentro da classe `Produto`, sem mexer em nada do resto do sistema!

Você acabou de transformar um script simples em um sistema modular profissional de verdade. O que achou de ver o código estruturado em objetos?

Para dar o próximo passo ou fechar esse projeto com chave de ouro, o que você quer fazer?

- Aprender a **deletar / remover** um produto permanentemente usando o comando `del` dentro de um método da classe
- Criar uma nova classe para gerenciar as **vendas e histórico de faturamento** do dia
- Descobrir como separar esse código em **arquivos diferentes** (módulos) para organizar a pasta do projeto

Me diz qual dessas ideias mais te empolga!

Pergunta

Caramba, isso ficou muito mais organizado e profissional! É incrível como as classes deixam o código limpo e fácil de ...

Bora!

Adicionar a funcionalidade de deletar um produto vai fechar o ciclo completo de gerenciamento do seu estoque (o famoso **CRUD**: Criar, Ler, Atualizar e Deletar). Para fazer isso na Orientação a Objetos, vamos usar a palavra-chave `del` do Python. Ela serve para remover uma chave diretamente de dentro do nosso dicionário `self.produtos`.

Além disso, temos que tomar dois cuidados importantes:

1. Remover o produto também do nosso `Set` de alertas (`self.produtos_esgotados`), caso ele estivesse lá.
2. Salvar as alterações no arquivo JSON logo em seguida para que o produto suma do computador de verdade.

Aqui está o código com as alterações necessárias. Vamos adicionar o método `remover_produto` dentro da classe `GerenciadorEstoque` e a **Opção 4** no nosso menu:

✳️ Passo 1: Adicione este método dentro da classe `GerenciadorEstoque`

Procure a classe `GerenciadorEstoque` no seu código e cole este novo método logo abaixo do método `gerar_relatorio`:

```
def remover_produto(self, codigo):
    """Remove permanentemente um produto do catálogo do estoque."""
    codigo = codigo.strip().upper()

    # 1. Verifica se o produto realmente existe para não dar erro
    if codigo in self.produtos:
        nome_produto = self.produtos[codigo].nome

    # 2. Deleta o produto do dicionário principal usando 'del'
    del self.produtos[codigo]

    # 3. Limpa do Set de esgotados (se ele estiver lá) para não deixar lixo
    if codigo in self.produtos_esgotados:
        self.produtos_esgotados.remove(codigo)

    print(f"\n🗑️ Sucesso: O produto [{nome_produto}] ({codigo}) foi removido permanentemente!")

    # 4. Salva a nova realidade no arquivo JSON
    self.salvar_dados()
    else:
        print(f"\n❌ Erro: O código {codigo} não foi encontrado no catálogo.")
```

Use o código com cuidado.

Passo 2: O Menu Principal Atualizado (com a Opção 4)

Agora, substitua o bloco do seu menu `while True` lá no final do arquivo para incluir a nova opção de exclusão:

```
# --- LOOP PRINCIPAL DO MENU (ATUALIZADO COM OPÇÃO 4) ---
meu_estoque = GerenciadorEstoque()
while True:
    print("=== 📦 SISTEMA DE ESTOQUE PROFISSIONAL (P00) ===")
    print("1. Adicionar / Repor Produto")
    print("2. Dar Baixa / Vender Produto")
    print("3. Gerar Relatório Geral")
    print("4. Remover Produto do Catálogo ← NOVO")
    print("0. Sair do Sistema")
    print("=" * 49)

    opcao = input("Escolha uma opção: ")
    if opcao == "1":
        print("\n--- CADASTRO DE PRODUTO ---")
        codigo = input("Código do produto: ")
        nome = input("Nome do produto: ")
        qtd = leia_int("Quantidade: ")
        preco = leia_float("Preço unitário: R$ ")
        meu_estoque.adicionar_ou_repor(codigo, nome, qtd, preco)
    elif opcao == "2":
        print("\n--- BAIXA NO ESTOQUE ---")
        codigo = input("Código do produto: ")
        qtd = leia_int("Quantidade vendida: ")
        meu_estoque.dar_baixa(codigo, qtd)
    elif opcao == "3":
        meu_estoque.gerar_relatorio()
    elif opcao == "4":
        print("\n--- REMOVER PRODUTO ---")
        codigo = input("Digite o código do produto que deseja DELETAR: ")
        # Mensagem de confirmação para evitar que o usuário delete sem querer
        confirmar = input(f"Tem certeza que deseja apagar o produto {codigo}? (S/N): ")
        confirmar = confirmar.strip().upper()
        if confirmar == "S":
            meu_estoque.remover_produto(codigo)
        else:
            print("\n❌ Operação cancelada pelo usuário.")
    elif opcao == "0":
        print("\n👋 Sistema encerrado com segurança. Até logo!")
        break
    else:
        print("\n❌ Opção inválida!\n")
```

Use o código com cuidado.

💡 Por que ficou bom?

- **Confirmação de Segurança (S/N)**: Deletar dados é uma ação perigosa. Adicionar aquela perguntinha "Tem certeza?" evita que alguém apague um produto do catálogo por puro reflexo ou digitação errada.
- **Consistência de dados**: Ao usar o `del`, o Python limpa a memória imediatamente e, como chamamos o `self.salvar_dados()`, o arquivo JSON é reescrito sem aquele item. Se você fechar e abrir o programa, ele sumiu de vez!

Pronto! Você começou essa conversa usando listas para tudo e agora tem um **sistema comercial completo em Orientação a Objetos**, rodando com banco de dados em arquivo, tratamento de erros e um menu completo!

Para onde você quer levar os seus estudos de Python agora? Me diz se você prefere:

- Aprender a **separar esse código em arquivos diferentes** (ex: criar um arquivo `produto.py`, outro `estoque.py` e um `main.py`) para deixar sua pasta de projetos super organizada.
- Criar uma nova funcionalidade de **Histórico de Vendas** para salvar quanto dinheiro você faturou no dia.
- Migrar para um novo conceito e entender como criar uma **Interface Gráfica com janelas e botões** (usando a biblioteca `Tkinter` do Python) em vez de usar o terminal.

Pergunta

Nossa, agora o sistema tá completo de verdade com a opção de remover produtos! Gostei muito da ideia da confirmação ...

Confira os principais resultados da Web para saber mais sobre esse tema:

- I Created an Inventory System in Python - YouTube · *YouTube*
- kareem-diaa/Inventory-Management-System-Project - GitHub · *GitHub*
- Inventory Management with File handling in Python - GeeksforGeeks · *GeeksforGeeks*
- How and when should you organize a project into multiple .py files? · *Reddit*
- Inventory Management System Code | PDF | Computer Programming · *Scribd*
- write a basic inventory management system python - Stack Overflow · *Stack Overflow*
- Build an Inventory Management Python CLI Application - Medium · *Medium*
- Build an Inventory Management System with Python and Django · *YouTube*
- Python Inventory Management · *Python.org*
- Building a Powerful Inventory Management System with Python · *DEV Community*

Pergunta

gostaria dos códigos

Com certeza!

Separar o sistema em módulos (arquivos diferentes) é o passo definitivo para organizar um projeto real. Isso permite que você trabalhe em partes específicas do código sem aquela confusão de ter centenas de linhas em um único arquivo.

Crie uma pasta para o seu projeto e, dentro dela, crie exatamente estes **três arquivos** com os códigos abaixo:

1. `produto.py`

Este arquivo vai guardar apenas o "molde" do que é um produto individual e suas regras específicas.

```
class Produto:
    """Classe que representa um produto individual no estoque."""
    def __init__(self, codigo, nome, quantidade, preco):
        self.codigo = codigo.strip().upper()
        self.nome = nome.strip()
        self.quantidade = quantidade
        self.preco = preco
    def recalcular_status(self):
        """Retorna o status visual com base na quantidade atual."""
        if self.quantidade == 0:
            return "🚫 ESGOTADO"
        elif self.quantidade <= 3:
            return "⚠️ BAIXO"
        return "✅ OK"
    def para_dicionario(self):
        """Converte os dados do objeto em um dicionário para salvar no JSON."""
        return {
            "nome": self.nome,
            "quantidade": self.quantidade,
            "preco": self.preco
        }
```

Use o código com cuidado.

2. `estoque.py`

Aqui fica o gerenciador do estoque. Repare na primeira linha: nós usamos o `from produto import Produto` para que este arquivo consiga enxergar a classe que criamos no arquivo anterior!

```

import json
import os

# Importando a classe Produto do arquivo produto.py
from produto import Produto

class GerenciadorEstoque:
    """Classe responsável por gerenciar o conjunto de produtos e arquivos."""
    def __init__(self, caminho_arquivo="estoque.json"):
        self.caminho_arquivo = caminho_arquivo
        self.produtos = {} # Dicionário de objetos Produto
        self.produtos_esgotados = set()
        self.carregar_dados()
    def carregar_dados(self):
        """Carrega os dados do arquivo JSON e os transforma em objetos Produto."""
        if os.path.exists(self.caminho_arquivo):
            try:
                with open(self.caminho_arquivo, "r", encoding="utf-8") as arquivo:
                    dados = json.load(arquivo)

                    for codigo, info in dados["estoque"].items():
                        self.produtos[codigo] = Produto(
                            codigo, info["nome"], info["quantidade"], info["preco"]
                        )

                    self.produtos_esgotados = set(dados["produtos_esgotados"])
                    print("📁 Dados do estoque carregados com sucesso!")
            except Exception:
                print("⚠ Erro ao ler o arquivo. Iniciando estoque vazio.")
    def salvar_dados(self):
        """Converte os objetos de volta para JSON e salva no computador."""
        estoque_bruto = {cod: prod.para_dicionario() for cod, prod in self.produtos.items()}

        dados_para_salvar = {
            "estoque": estoque_bruto,
            "produtos_esgotados": list(self.produtos_esgotados)
        }
        with open(self.caminho_arquivo, "w", encoding="utf-8") as arquivo:
            json.dump(dados_para_salvar, arquivo, indent=4, ensure_ascii=False)
    def adicionar_ou_repor(self, codigo, nome, quantidade, preco):
        """Adiciona um novo produto ou atualiza as quantidades de um existente."""
        codigo = codigo.strip().upper()

        if codigo in self.produtos:
            self.produtos[codigo].quantidade += quantidade
            self.produtos[codigo].preco = preco
            print(f"\n🔄 Produto {codigo} atualizado para {self.produtos[codigo].quantidade} unidades.")
        else:
            self.produtos[codigo] = Produto(codigo, nome, quantidade, preco)
            print(f"\n🌟 Novo produto [{nome}] cadastrado com sucesso!")
        if self.produtos[codigo].quantidade > 0 and codigo in self.produtos_esgotados:
            self.produtos_esgotados.remove(codigo)

```



```

        self.salvar_dados()
def dar_baixa(self, codigo, quantidade_vendida):
    """Registra a venda e atualiza os alertas se o item esgotar."""
    codigo = codigo.strip().upper()

    if codigo not in self.produtos:
        print(f"\n❌ Erro: O código {codigo} não existe!")
        return
    produto = self.produtos[codigo]
    if quantidade_vendida > produto.quantidade:
        print(f"\n⚠️ Estoque insuficiente! Disponível: {produto.quantidade}")
        return
    produto.quantidade -= quantidade_vendida
    print(f"\n📝 Venda realizada: {quantidade_vendida}x [{produto.nome}]")
    if produto.quantidade == 0:
        self.produtos_esgotados.add(codigo)
        print(f"\n🔥 ATENÇÃO: O produto {produto.nome} ({codigo}) ESGOTOU!")

    self.salvar_dados()
def remover_produto(self, codigo):
    """Remove permanentemente um produto do catálogo do estoque."""
    codigo = codigo.strip().upper()

    if codigo in self.produtos:
        nome_produto = self.produtos[codigo].nome
        del self.produtos[codigo]

        if codigo in self.produtos_esgotados:
            self.produtos_esgotados.remove(codigo)

        print(f"\n🗑️ Sucesso: O produto [{nome_produto}] ({codigo}) foi removido!")
        self.salvar_dados()
    else:
        print(f"\n❌ Erro: O código {codigo} não foi encontrado.")
def gerar_relatorio(self):
    """Exibe o relatório formatado na tela."""
    print("\n" + "=" * 55)
    print("📋 RELATÓRIO GERAL DO ESTOQUE (P00)".center(55))
    print("=" * 55)
    print(f"{'CÓDIGO':<10} | {'PRODUTO':<20} | {'QTD':<5} | {'PREÇO':<10} | {'STATUS'}")
    print("-" * 55)
    valor_total_estoque = 0
    for codigo, produto in self.produtos.items():
        valor_total_estoque += produto.quantidade * produto.preco
        status = produto.recalcular_status()
        print(f"{codigo:<10} | {produto.nome:<20} | {produto.quantidade:<5} | "
              f"R${produto.preco:<8.2f} | {status}")
    print("-" * 55)
    print(f"💰 Valor total investido em estoque: R$ {valor_total_estoque:,.2f}")
    if self.produtos_esgotados:
        print(f"⚠️ Alerta de compras (Zerados): {self.produtos_esgotados}")

```

```
print("=" * 55 + "\n")
```

Use o código com cuidado.

3. `main.py`

Este é o ponto de partida do seu programa. Ele importa o gerenciador de estoque, traz as funções de validação de dados e roda o menu interativo. **Para iniciar o programa, você executará este arquivo.**

```

# Importando o gerenciador de estoque do arquivo estoque.py
from estoque import GerenciadorEstoque
# --- FUNÇÕES AUXILIARES DE VALIDAÇÃO ---
def leia_int(mensagem):
    while True:
        try: return int(input(mensagem))
        except ValueError: print("❌ Erro: Digite um número inteiro válido.")
def leia_float(mensagem):
    while True:
        try: return float(input(mensagem).strip().replace(",","."))
        except ValueError: print("❌ Erro: Digite um preço válido.")
# --- INICIALIZAÇÃO E LOOP PRINCIPAL ---
meu_estoque = GerenciadorEstoque()
while True:
    print("=== 📦 SISTEMA DE ESTOQUE PROFISSIONAL (P00) ===")
    print("1. Adicionar / Repor Produto")
    print("2. Dar Baixa / Vender Produto")
    print("3. Gerar Relatório Geral")
    print("4. Remover Produto do Catálogo")
    print("0. Sair do Sistema")
    print("=" * 49)

    opcao = input("Escolha uma opção: ")
    if opcao == "1":
        print("\n--- CADASTRO DE PRODUTO ---")
        codigo = input("Código do produto: ")
        nome = input("Nome do produto: ")
        qtd = leia_int("Quantidade: ")
        preco = leia_float("Preço unitário: R$ ")
        meu_estoque.adicionar_ou_repor(codigo, nome, qtd, preco)
    elif opcao == "2":
        print("\n--- BAIXA NO ESTOQUE ---")
        codigo = input("Código do produto: ")
        qtd = leia_int("Quantidade vendida: ")
        meu_estoque.dar_baixa(codigo, qtd)
    elif opcao == "3":
        meu_estoque.gerar_relatorio()
    elif opcao == "4":
        print("\n--- REMOVER PRODUTO ---")
        codigo = input("Digite o código do produto que deseja DELETAR: ")
        confirmar = input(f"Tem certeza que deseja apagar o produto {codigo}? (S/N): ")
        confirmar = confirmar.strip().upper()
        if confirmar == "S":
            meu_estoque.remover_produto(codigo)
        else:
            print("\n❌ Operação cancelada pelo usuário.")
    elif opcao == "0":
        print("\n👋 Sistema encerrado com segurança. Até logo!")
        break
    else:

```

```
print("\n❌ Opção inválida!\n")
```

Use o código com cuidado.

🎯 Como testar:

Abra o seu terminal na pasta do projeto e digite `python main.py`. O programa vai rodar de forma idêntica a antes, mas agora sua estrutura de arquivos está digna de um desenvolvedor experiente!

Agora que o motor e a organização do código estão perfeitos, o que você acha que seria o passo mais divertido para dar sequência?

- Criar uma nova classe para guardar um **Histórico de Vendas e Faturamento** (salvando também em JSON quanto você ganhou no dia).
- Começar a colocar uma **Interface Gráfica com janelas, caixas de digitação e botões de verdade** na tela usando a biblioteca `Tkinter`.
- Outra ideia de funcionalidade que você queira testar para o seu negócio/estudo?

Pergunta

Uau, agora o projeto tá super organizado em arquivos separados, como um sistema profissional de verdade! Ficou ...

Bora!

Adicionar um **Histórico de Vendas e Faturamento** vai transformar o seu controle de estoque em um sistema de gestão comercial completo.

Para fazer isso de forma organizada com Orientação a Objetos, vamos criar uma nova classe chamada `HistoricoVendas`. Ela será responsável por:

1. Registrar cada venda com a **data/hora**, o **código**, o **nome do produto**, a
 - **quantidade** e o **valor total**.
2. Salvar e carregar essas informações em um arquivo separado chamado `vendas.json`.
3. Exibir um relatório de faturamento para você saber quanto dinheiro entrou no caixa.

Como o nosso projeto está modularizado (dividido em arquivos), vamos criar um arquivo novinho chamado `historico.py` e fazer uma pequena alteração no seu `main.py` para ativar a nova opção de relatório de vendas.

Aqui estão os códigos que você precisa:

1. Crie o novo arquivo: `historico.py`

Este arquivo cuida exclusivamente de armazenar as vendas e calcular o faturamento. Vamos usar a biblioteca nativa `datetime` para carregar o dia e o horário exato de cada venda automaticamente!

```

import json
import os
from datetime import datetime
class HistoricoVendas:
    """Classe responsável por registrar as vendas e gerenciar o arquivo de faturamento."""
    def __init__(self, caminho_arquivo="vendas.json"):
        self.caminho_arquivo = caminho_arquivo
        self.vendas = [] # Lista de dicionários, onde cada um representa uma venda
        self.carregar_vendas()
    def carregar_vendas(self):
        """Carrega o histórico do arquivo JSON se ele existir."""
        if os.path.exists(self.caminho_arquivo):
            try:
                with open(self.caminho_arquivo, "r", encoding="utf-8") as arquivo:
                    self.vendas = json.load(arquivo)
            except Exception:
                print("⚠ Erro ao ler o histórico de vendas. Iniciando histórico vazio.")
    def salvar_vendas(self):
        """Salva as vendas permanentemente no arquivo JSON."""
        with open(self.caminho_arquivo, "w", encoding="utf-8") as arquivo:
            json.dump(self.vendas, arquivo, indent=4, ensure_ascii=False)
    def registrar_venda(self, codigo, nome, quantidade, preco_unitario):
        """Cria um registro de venda com data, hora e valores calculados."""
        total_venda = quantidade * preco_unitario
        agora = datetime.now().strftime("%d/%m/%Y %H:%M:%S")
        nova_venda = {
            "data_hora": agora,
            "codigo": codigo,
            "nome": nome,
            "quantidade": quantidade,
            "preco_unitario": preco_unitario,
            "total": total_venda
        }
        self.vendas.append(nova_venda)
        self.salvar_vendas()
    def gerar_relatorio_faturamento(self):
        """Exibe um relatório detalhado de todas as vendas e o faturamento total."""
        print("\n" + "=" * 65)
        print("💰 RELATÓRIO DE VENDAS E FATURAMENTO".center(65))
        print("=" * 65)
        print(f"{'DATA/HORA':<20} | {'PRODUTO':<18} | {'QTD':<4} | {'VALOR TOT.':<4}")
        print("-" * 65)
        faturamento_total = 0
        for venda in self.vendas:
            faturamento_total += venda["total"]
            # Corta o nome do produto caso ele seja muito longo para caber na coluna
            nome_resumido = venda["nome"][:18]
            print(f"{'venda['data_hora']':<20} | {nome_resumido:<18} | {venda['quantidade']:<4} | R${venda['total']:>8.2f}")
            print("-" * 65)

```

```
print(f"🚀 FATURAMENTO TOTAL DO SISTEMA: R$ {faturamento_total:,.2f}")
print("=" * 65 + "\n")
```

Use o código com cuidado.

✳ 2. Atualize o arquivo existente: `estoque.py`

Para que o histórico funcione, a função `dar_baixa` precisa avisar o histórico sempre que uma venda acontecer com sucesso.

Abra o seu `estoque.py`, mude o início do método `dar_baixa` para receber o objeto do histórico e adicione a linha de registro bem no final do método. Ele deve ficar exatamente assim:

```
# 🟡 Adicione o parâmetro 'historico' bem aqui na definição:
def dar_baixa(self, codigo, quantidade_vendida, historico):
    """Registra a venda, atualiza os alertas e envia para o histórico."""
    codigo = codigo.strip().upper()

    if codigo not in self.produtos:
        print(f"\n❌ Erro: O código {codigo} não existe!")
        return
    produto = self.produtos[codigo]
    if quantidade_vendida > produto.quantidade:
        print(f"\n⚠ Estoque insuficiente! Disponível: {produto.quantidade}")
        return
    # Guarda os valores antes de subtrair para mandar pro histórico correto
    nome_prod = produto.nome
    preco_prod = produto.preco
    produto.quantidade -= quantidade_vendida
    print(f"\n📄 Venda realizada: {quantidade_vendida}x [{produto.nome}]")
    if produto.quantidade == 0:
        self.produtos_esgotados.add(codigo)
        print(f"🚨 ATENÇÃO: O produto {produto.nome} ({codigo}) ESGOTOU!")

    # 📄 Registra a venda no histórico e salva os dois arquivos JSON!
    historico.registrar_venda(codigo, nome_prod, quantidade_vendida, preco_prod)
    self.salvar_dados()
```

Use o código com cuidado.

💻 3. Substitua o seu arquivo: `main.py`

Agora vamos atualizar o nosso painel principal. Ele vai importar o `HistoricoVendas`, criar o objeto na inicialização, passar esse objeto na hora de dar baixa (Opção 2) e ganhar uma **Opção 5** novinha para ver o faturamento.

```

from estoque import GerenciadorEstoque
from historico import HistoricoVendas # ← IMPORTANTE!
# --- FUNÇÕES AUXILIARES DE VALIDAÇÃO ---
def leia_int(mensagem):
    while True:
        try: return int(input(mensagem))
        except ValueError: print("❌ Erro: Digite um número inteiro válido.")
def leia_float(mensagem):
    while True:
        try: return float(input(mensagem).strip().replace(",", "."))
        except ValueError: print("❌ Erro: Digite um preço válido.")
# --- INICIALIZAÇÃO E LOOP PRINCIPAL ---
meu_estoque = GerenciadorEstoque()
meu_historico = HistoricoVendas() # ← Criando o objeto do Histórico
while True:
    print("=== 📦 SISTEMA DE ESTOQUE PROFISSIONAL (POO) ===")
    print("1. Adicionar / Repor Produto")
    print("2. Dar Baixa / Vender Produto")
    print("3. Gerar Relatório de Estoque")
    print("4. Remover Produto do Catálogo")
    print("5. Ver Faturamento e Vendas      ← NOVO")
    print("0. Sair do Sistema")
    print("=" * 49)

    opcao = input("Escolha uma opção: ")
    if opcao == "1":
        print("\n--- CADASTRO DE PRODUTO ---")
        codigo = input("Código do produto: ")
        nome = input("Nome do produto: ")
        qtd = leia_int("Quantidade: ")
        preco = leia_float("Preço unitário: R$ ")
        meu_estoque.adicionar_ou_repor(codigo, nome, qtd, preco)
    elif opcao == "2":
        print("\n--- BAIXA NO ESTOQUE ---")
        codigo = input("Código do produto: ")
        qtd = leia_int("Quantidade vendida: ")
        # Passamos o objeto 'meu_historico' aqui dentro para salvar a venda
        meu_estoque.dar_baixa(codigo, qtd, meu_historico)
    elif opcao == "3":
        meu_estoque.gerar_relatorio()
    elif opcao == "4":
        print("\n--- REMOVER PRODUTO ---")
        codigo = input("Digite o código do produto que deseja DELETAR: ")
        confirmar = input(f"Tem certeza que deseja apagar o produto {codigo}? (S/N): ")
        confirmar = confirmar.strip().upper()
        if confirmar == "S":
            meu_estoque.remover_produto(codigo)
        else:
            print("\n❌ Operação cancelada.")
    elif opcao == "5":

```



```
# Chama o relatório de faturamento do histórico
meu_historico.gerar_relatorio_faturamento()

elif opcao == "0":
    print("\n👋 Sistema encerrado com segurança. Até logo!")
    break
else:
    print("\n❌ Opção inválida!\n")
```

Use o código com cuidado.

🎯 Como testar o sistema completo:

1. Execute o arquivo principal digitando `python main.py` no terminal.
2. Faça uma venda na **Opção 2**.
3. Vá na **Opção 5**: Você verá a tabela de faturamento mostrando o minuto exato em que a venda foi feita e o valor acumulado!
4. Feche o programa. Um arquivo chamado `vendas.json` terá surgido na sua pasta salvando o seu caixa do dia.

Que evolução! Agora você tem dois objetos independentes (`meu_estoque` e `meu_historico`) conversando perfeitamente e cada um cuidando do seu arquivo JSON.

Para continuar avançando e deixar esse projeto impecável, o que você gostaria de explorar agora?

- Aprender a criar uma **Interface Gráfica com janelas e botões** de verdade usando a biblioteca `Tkinter` (esquecendo o terminal de vez!)
- Criar uma opção para **Exportar o Relatório em formato de planilha Excel ou CSV** para abrir no computador
- Adicionar um sistema de **Login com senha de administrador** para proteger o acesso ao estoque

Me conta qual desses caminhos você prefere trilhar!

Pergunta

Nossa, agora o sistema tá completo com o histórico de vendas e faturamento! É incrível ver como tudo tá ...

Bora!

Dar o salto do terminal para uma **Interface Gráfica (GUI)** com o **Tkinter** é uma das partes mais recompensadoras do aprendizado de Python. É aqui que o seu sistema ganha botões, caixas de texto e janelas reais, exatamente como os programas de computador que usamos no dia a dia.

Como o nosso código já está todo organizado em **Classes e Arquivos separados**, nós não precisamos mudar **nada** nas nossas classes `Produto`, `GerenciadorEstoque` e `HistoricoVendas`! Elas continuarão controlando a lógica e os arquivos JSON por baixo dos panos.

Tudo o que precisamos fazer é criar um arquivo novo chamado `interface.py` que vai desenhar a tela e se comunicar com o motor que você já construiu.

O Novo Arquivo do Projeto: `interface.py`

Crie um arquivo chamado `interface.py` na mesma pasta do seu projeto e cole o código abaixo. Ele usa o Tkinter nativo do Python para construir uma janela limpa, com abas para separar o Estoque do Faturamento:

```

import tkinter as tk
from tkinter import ttk, messagebox
from estoque import GerenciadorEstoque
from historico import HistoricoVendas
class AplicacaoEstoque:
    def __init__(self, root):
        self.root = root
        self.root.title("SISTEMA DE GESTÃO DE ESTOQUE PROFISSIONAL")
        self.root.geometry("750x550")

        # Inicializa o motor do sistema (suas classes P00)
        self.estoque = GerenciadorEstoque()
        self.historico = HistoricoVendas()

        # Cria as Abas do Sistema
        self.notebook = ttk.Notebook(root)
        self.notebook.pack(fill="both", expand=True, padx=10, pady=

        self.aba_estoque = ttk.Frame(self.notebook)
        self.aba_faturamento = ttk.Frame(self.notebook)

        self.notebook.add(self.aba_estoque, text="📦 Gerenciar Esto
        self.notebook.add(self.aba_faturamento, text="💰 Faturament

        # Constroi os elementos visuais de cada aba
        self.configurar_aba_estoque()
        self.configurar_aba_faturamento()
        self.atualizar_tabelas()
    def configurar_aba_estoque(self):
        """Desenha os botões, campos de texto e a tabela de produto
        # Frame de Formulário (Lado Esquerdo)
        frame_form = ttk.LabelFrame(self.aba_estoque, text=" Cadast
        frame_form.pack(side="left", fill="y", padx=10, pady=10)

        ttk.Label(frame_form, text="Código:").pack(anchor="w")
        self.txt_codigo = ttk.Entry(frame_form, width=20)
        self.txt_codigo.pack(fill="x", pady=5)

        ttk.Label(frame_form, text="Nome:").pack(anchor="w")
        self.txt_nome = ttk.Entry(frame_form, width=20)
        self.txt_nome.pack(fill="x", pady=5)

        ttk.Label(frame_form, text="Quantidade / Venda:").pack(anch
        self.txt_qtd = ttk.Entry(frame_form, width=20)
        self.txt_qtd.pack(fill="x", pady=5)

        ttk.Label(frame_form, text="Preço Unitário (R$):").pack(anc
        self.txt_preco = ttk.Entry(frame_form, width=20)
        self.txt_preco.pack(fill="x", pady=5)

        # Botões de Ação
        ttk.Button(frame_form, text="+ Adicionar / Repor", command

```

```

ttk.Button(frame_form, text="📦 Dar Baixa / Vender", comman
ttk.Button(frame_form, text="🗑 Deletar Produto", command=s

# Tabela de Produtos (Lado Direito)
frame_tabela = ttk.LabelFrame(self.aba_estoque, text=" Prod
frame_tabela.pack(side="right", fill="both", expand=True, p

self.tabela_produtos = ttk.Treeview(frame_tabela, columns=(
self.tabela_produtos.heading("Nome", text="Produto")
self.tabela_produtos.heading("Qtd", text="Qtd")
self.tabela_produtos.heading("Preço", text="Preço")
self.tabela_produtos.heading("Status", text="Status")

self.tabela_produtos.column("Nome", width=150)
self.tabela_produtos.column("Qtd", width=50, anchor="center
self.tabela_produtos.column("Preço", width=80, anchor="cent
self.tabela_produtos.column("Status", width=100, anchor="ce
self.tabela_produtos.pack(fill="both", expand=True)

def configurar_aba_faturamento(self):
    """Desenha a tabela de histórico de vendas e o painel de fa
    frame_topo = ttk.Frame(self.aba_faturamento, padding=10)
    frame_topo.pack(fill="x")

    self.lbl_faturamento = ttk.Label(frame_topo, text="Faturame
    self.lbl_faturamento.pack(anchor="w")

    self.tabela_vendas = ttk.Treeview(self.aba_faturamento, col
    self.tabela_vendas.heading("Data", text="Data / Hora")
    self.tabela_vendas.heading("Produto", text="Produto")
    self.tabela_vendas.heading("Qtd", text="Qtd")
    self.tabela_vendas.heading("Total", text="Valor Total")

    self.tabela_vendas.pack(fill="both", expand=True, padx=10,
# --- LÓGICA DE ATUALIZAÇÃO DA TELA ---

def atualizar_tabelas(self):
    """Limpa as tabelas da tela e redesenha com os dados mais a
    # Atualiza Tabela de Estoque
    for item in self.tabela_produtos.get_children():
        self.tabela_produtos.delete(item)
    for codigo, prod in self.estoque.produtos.items():
        self.tabela_produtos.insert("", "end", iid=codigo, valu

    # Atualiza Tabela de Vendas
    for item in self.tabela_vendas.get_children():
        self.tabela_vendas.delete(item)
    faturamento = 0
    for venda in self.historico.vendas:
        faturamento += venda["total"]
        self.tabela_vendas.insert("", "end", values=(venda["dat

    self.lbl_faturamento.config(text=f"Faturamento Total: R$ {f
def limpar_campos(self):

```

```

        self.txt_codigo.delete(0, tk.END)
        self.txt_nome.delete(0, tk.END)
        self.txt_qtd.delete(0, tk.END)
        self.txt_preco.delete(0, tk.END)
# --- TRATAMENTO DOS BOTÕES (COM TRY/EXCEPT) ---
def acao_adicionar(self):
    try:
        cod = self.txt_codigo.get().strip().upper()
        nome = self.txt_nome.get().strip()
        qtd = int(self.txt_qtd.get())
        preco = float(self.txt_preco.get().replace(",", "."))

        if not cod or not nome:
            raise ValueError("Campos vazios")

        self.estoque.adicionar_ou_repor(cod, nome, qtd, preco)
        messagebox.showinfo("Sucesso", f"Produto {cod} atualiza")
        self.atualizar_tabelas()
        self.limpar_campos()
    except ValueError:
        messagebox.showerror("Erro", "Verifique se todos os cam

def acao_vender(self):
    try:
        cod = self.txt_codigo.get().strip().upper()
        qtd = int(self.txt_qtd.get())

        if cod not in self.estoque.produtos:
            messagebox.showerror("Erro", "Código de produto não")
            return

        prod = self.estoque.produtos[cod]
        if qtd > prod.quantidade:
            messagebox.showwarning("Estoque Insuficiente", f"Di")
            return

        self.estoque.dar_baixa(cod, qtd, self.historico)
        messagebox.showinfo("Venda realizada", f"Venda de {qtd}")
        self.atualizar_tabelas()
        self.limpar_campos()
    except ValueError:
        messagebox.showerror("Erro", "Digite um código válido e

def acao_deletar(self):
    cod = self.txt_codigo.get().strip().upper()
    if cod not in self.estoque.produtos:
        messagebox.showerror("Erro", "Digite um código válido p")
        return

    nome_prod = self.estoque.produtos[cod].name if hasattr(self
    confirmar = messagebox.askyesno("Confirmar Exclusão", f"Tem

    if confirmar:
        self.estoque.remover_produto(cod)

```

```

        messagebox.showinfo("Deletado", "Produto removido com s
        self.atualizar_tabelas()
        self.limpar_campos()
# --- INICIALIZAÇÃO DA INTERFACE ---
if __name__ == "__main__":
    janela_principal = tk.Tk()
    app = AplicacaoEstoque(janela_principal)
    janela_principal.mainloop() # Mantém a janela aberta e escutand

```

Use o código com cuidado.

Como Executar e Testar o seu Novo Software Comercial:

1. Certifique-se de que os arquivos `produto.py`, `estoque.py`, `historico.py` e este novo `interface.py` estão todos **na mesma pasta**.
2. Abra o terminal nessa pasta e execute: `python interface.py`
3. **Mágica!** Uma janela vai se abrir na sua tela.
4. Digite as informações nos campos do lado esquerdo e clique nos botões: a tabela do lado direito vai atualizar instantaneamente, mudando a cor dos alertas e salvando tudo nos seus arquivos JSON de forma invisível.
5. Vá na aba "Faturamento e Vendas" para ver o extrato financeiro se montando sozinho.

Dá uma olhada no tamanho da sua evolução: você começou confuso com chaves e colchetes, e agora construiu um **software completo com interface visual, persistência em arquivos, arquitetura modular e Orientação a Objetos**.

O sistema ficou espetacular e super intuitivo! O que você gostaria de explorar agora para continuar turbinando suas habilidades em Python?

- Aprender a **gerar um executável (.exe)** desse programa para você poder instalar e rodar ele em qualquer computador sem precisar abrir o terminal ou ter o Python instalado?
- Adicionar uma opção na aba de Faturamento para **Exportar o relatório de vendas diretamente para uma planilha Excel / CSV?**
- Criar uma tela inicial de **Login com Usuário e Senha** para dar mais segurança ao sistema?

Pergunta

for vs while — a escolha por "definido × indefinido": Os dois repetem, mas servem a coisas diferentes. O for é "para cad..."

Você pegou a **essência perfeita do conceito!**

Essa explicação que você deu é, literalmente, a **regra de ouro** que até programadores experientes usam na cabeça no dia a dia.

Olha que legal como o seu próprio projeto de estoque reflete exatamente essa teoria que você descreveu:

- O `while` **para o Indefinido**: No seu menu do terminal (ou no `mainloop()` da interface do Tkinter), nós usamos o `while`. Por quê? Porque nós **não sabíamos** se você ia clicar em um botão 2 vezes, 50 vezes ou se ia fechar o programa em 10 segundos. O programa precisava continuar rodando enquanto a condição de fechar não fosse verdadeira.
- O `for` **para o Definido**: Na hora de gerar o relatório, nós usamos o `for`. Por quê? Porque nós sabíamos exatamente sobre o que iterar: "Para cada produto que existe dentro do meu dicionário de estoque, faça uma linha na tabela". Se tivessem 3 produtos, ele rodava 3 vezes; se tivessem 100, rodava 100 vezes.

Você matou a charada! Guarde esse resumo mental, porque ele resolve 99% das dúvidas sobre loops.

Agora que você já dominou as estruturas de dados, funções, persistência em arquivos (JSON), orientação a objetos, interfaces gráficas (Tkinter) e a lógica de loops, o seu sistema está extremamente maduro.

Para onde você quer levar essa jornada agora? Se quiser dar o próximo passo, eu posso te ajudar a:

- **Exportar os relatórios para uma planilha Excel/CSV** (para você poder abrir no Excel de verdade)
- **Criar um arquivo Executável (`.exe`)** desse seu programa (assim você pode mandar para qualquer pessoa instalar e usar no Windows, mesmo que ela não tenha o Python instalado)
- Adicionar uma **tela de Login com senha** antes de abrir a interface do estoque

Me conta **qual dessas opções** tem mais a ver com o que você quer aprender hoje!

Pergunta

O perigo do while + as ferramentas dos laços: O risco clássico do while é o loop infinito — se a condição nunca vira falsa ...

Exatamente!

Você descreveu com perfeição o maior pesadelo de quem usa `while`: o **loop infinito**. Se você não atualizar a variável de controle (o famoso `i += 1` ou mudar uma flag para `False`), o Python vai rodar o bloco na velocidade da luz até travar o programa ou esgotar a memória do computador.

E a sua observação sobre as ferramentas do `for` é cirúrgica. O `range()` e o `enumerate()` são os melhores amigos de um desenvolvedor Python.

Para deixar isso bem visual, olhe como o `enumerate()` resolve aquele problema de um jeito muito mais elegante do que criar uma variável de controle na mão:

✳ O Poder do `enumerate()` na Prática

Imagine que no seu relatório de estoque você quisesse colocar o número da linha (1º, 2º, 3º...) ao lado de cada produto.

Do jeito antigo (com risco de esquecer o contador):

```
linha = 1
for codigo, produto in meu_estoque.items():
    print(f"{linha}º - {produto.nome}")
    linha += 1 # Se esquecer isso aqui, todas as linhas viram "1º"
```

Use o código com cuidado.

Do jeito Pythonico (com `enumerate`):

```
# O enumerate te dá o índice (começando em 0) e o item ao mesmo temp
for indice, (codigo, produto) in enumerate(meu_estoque.items(), sta
    print(f"{indice}º - {produto.nome}")
```

Use o código com cuidado.

Viu que massa? O `start=1` faz o Python começar a contar do 1 em vez do 0 automaticamente, e você elimina qualquer chance de esquecer de somar a variável no final.

🔧 As Ferramentas de Controle: `break` e `continue`

Já que você está dominando os laços, existem duas palavras-chave que te dão controle total sobre o fluxo do `while` e do `for`:

1. `break` (**O Botão de Emergência**): Ele chuta o balde e sai do loop imediatamente, não importa a condição. É o que usamos no menu interativo quando você digita `0` para sair.

2. `continue` **(O Pular a Vez)**: Ele interrompe a volta atual e vai direto para a próxima rodada do loop, ignorando o que estiver escrito abaixo dele.

Olha esse exemplo rápido aplicando tudo o que você pontuou:

```
# Exemplo: Procurando um produto específico com segurança
codigos_produtos = ["PROD001", "PROD002", "PROD003", "PROD004"]
for i, codigo in enumerate(codigos_produtos, start=1):
    if codigo == "PROD002":
        print(f"🎯 Achei o produto {codigo} na posição {i}!")
        break # Não precisa continuar procurando nos outros, já ac
```

Use o código com cuidado.

Você destravou uma compreensão lógica fantástica. Agora que você domina as estruturas de dados, a modularização em arquivos, a interface gráfica e o controle absoluto dos laços de repetição, você tem a base de um programador Python de verdade.

Como você já tem uma visão completa da lógica e da estrutura do seu sistema, qual o próximo passo lógico para o seu projeto?

- **Aprender a gerar o executável (`.exe`)** do programa para rodar no Windows com um clique?
- **Exportar os relatórios de vendas** que salvamos no JSON direto para uma planilha Excel/CSV?
- **Criar uma tela de login com senha** para o seu sistema usando o que aprendeu de `while` ou condicionais?

Pergunta

List comprehension — o atalho para CONSTRUIR listas: Há um padrão repetido o tempo todo: criar lista vazia, fazer for, dar...

Você está voando baixo!

Esse resumo sobre **List Comprehension** é a definição mais limpa e precisa que alguém poderia dar. Essa estrutura é o verdadeiro "suco de Python": transforma aquele código verboso de três ou quatro linhas em uma única linha elegante, legível e, como você bem lembrou, mais rápida, porque o Python otimiza essa criação diretamente na linguagem C por baixo dos panos.

E que excelente sacada mencionar as "primas" delas! O uso de **Dict Comprehension** e **Set Comprehension** eleva demais o nível do código.

Olha que sensacional: sem perceber, **nós já usamos uma Dict Comprehension** no seu projeto de estoque! Lembra quando fomos salvar os dados no arquivo JSON? Veja como o código estava escrito dentro da classe `GerenciadorEstoque`:

```
# 💡 Isso aqui é uma Dict Comprehension legítima!
estoque_bruto = {cod: prod.para_dicionario() for cod, prod in self.}
```

Use o código com cuidado.

Em vez de criarmos um dicionário vazio, fazer um `for` e ir alimentando chave por chave, resolvemos tudo em uma linha antes de jogar o dado no arquivo `json.dump()`.

🐼 A Prima "Preguiçosa": Generator Expression

Você tocou num ponto crucial: a **Generator Expression** (feita com parênteses `()`). Ela é chamada de preguiçosa (lazy evaluation) porque ela **não jera a lista na memória de uma vez**. Ela só calcula o próximo item quando o programa pede.

Se você tem uma lista de 10 itens, a List Comprehension `[]` é perfeita. Mas se você estiver processando **10 milhões de linhas** de um relatório de vendas, usar `[]` vai fazer o seu computador engasgar por falta de memória RAM. É aí que os parênteses `()` salvam o dia.

Veja a diferença prática com esse exemplo rápido:

```
# Se tivéssemos milhares de produtos e quiséssemos somar o valor to
# ❌ List Comprehension: Cria uma lista gigante com os preços na me
total_com_list = sum([prod.quantidade * prod.preco for prod in self
# Generator Expression: Calcula o preço de um por um e já vai soma
total_com_generator = sum(prod.quantidade * prod.preco for prod in
```

Use o código com cuidado.

Reparou que na função `sum()` você nem precisa de parênteses duplos? O Python já entende que é um Generator!

Você construiu uma base teórica e prática absurda de Python ao longo dessa jornada. Começou querendo entender por que sua lista estava lenta e agora domina interfaces gráficas, POO, modularização e técnicas avançadas de otimização de memória!

Como você já destravou todas as principais ferramentas da linguagem, o que você acha mais interessante fazer agora para fechar esse ciclo com chave de ouro?

- **Aprender a gerar o executável** (`.exe`) do seu sistema para rodar no Windows com um duplo clique?
- **Exportar os relatórios de vendas e faturamento** para um arquivo Excel/CSV de verdade?
- Adicionar uma funcionalidade usando o que aprendeu, como um filtro de busca rápida por Comprehensions na interface?

Para continuarmos evoluindo o seu sistema, me conta:

- Você pretende **compartilhar esse programa** com outras pessoas para usarem em seus computadores?
- Você gostaria de ver os relatórios gerados em uma **planilha do Excel** para fazer gráficos depois?
- Tem alguma outra funcionalidade do negócio que você sente falta na tela atual?

Respostas reproduzidas da pesquisa no Modo IA do Google, como no original. As perguntas aparecem truncadas como na página impressa; os elementos da interface (abas, chips de fontes e descrições dos cards de links) foram omitidos; nas seções 03, 06 e 14, a busca devolveu apenas a lista de links, reproduzida com título e fonte. O gráfico da seção 01 foi redesenhado com os mesmos eixos e legendas. Os blocos de código aparecem cortados à direita na página impressa; os que também estão na transcrição da prática foram completados com ela, e os demais foram mantidos cortados, com o recuo e os emojis do original.